

AWS 3-Tier Application Deployment Project Plan

Two Enterprise-Style Projects: EC2 and ECS + GitHub Actions CI/CD

This project plan is for two practical AWS deployment projects. Both projects use the same three-tier application model—React/Nginx frontend, FastAPI backend, and MySQL database—while comparing traditional EC2 deployment with containerized ECS deployment. The plan keeps the original source scope of application development, containerization, infrastructure as code, CI, security, and observability, but removes EKS/Argo CD and uses GitHub Actions for CI/CD. The source plan identifies React, FastAPI, MySQL, Terraform, GitHub Actions, security scanning, and observability as core elements.

1. Target Architecture

Tier	Application	AWS Deployment	Enterprise Practices
Presentation	ReactJS + Nginx	EC2 or ECS	ALB, TLS, private subnets where appropriate
Application	FastAPI REST API	EC2 or ECS	Auto scaling, health checks, least privilege
Data	MySQL	Prefer RDS MySQL	Private subnet, encryption, backups, controlled access

Enterprise baseline: deploy application workloads across multiple Availability Zones where supported; keep the database private; use IAM roles instead of long-lived AWS keys; encrypt data in transit and at rest; store secrets in AWS Secrets Manager or Systems Manager Parameter Store; use CloudWatch for logs/metrics; use Terraform for repeatable infrastructure.

2. Project 1 — 3-Tier Application on EC2

Objective: Build and deploy the application using EC2-based compute. The project demonstrates traditional VM operations, AMI/user-data based provisioning, Auto Scaling, ALB routing, and GitHub Actions deployment.

Phase 1 — Application Foundation

- Create React frontend, FastAPI backend, and MySQL data model.
- Implement CRUD APIs and /healthz endpoint.
- Use environment-based configuration.
- Run the complete stack locally with Docker Compose.

Phase 2 — AWS Foundation with Terraform

- Create VPC, public/private subnets, route tables, NAT/IGW as required.
- Create ALB, security groups, IAM roles, EC2 launch template and Auto Scaling Group.
- Create RDS MySQL in private subnets.
- Configure encrypted Terraform remote state and separate dev/prod variables.

Phase 3 — EC2 Application Packaging

- Create hardened EC2 application images or bootstrap instances with user data.
- Install Nginx and application runtime through automated provisioning.
- Run frontend and backend as managed services/processes.
- Configure CloudWatch Agent/logging and application health checks.

Phase 4 — GitHub Actions CI/CD

- Pull request: lint/test, build validation, Terraform fmt/validate, Checkov, Trivy, and code-quality checks.
- Main branch: build release artifact, publish versioned package/image if used, and deploy through an approved environment.

- Use GitHub OIDC to assume an AWS IAM deployment role instead of storing long-lived AWS access keys.
- Use environment approvals for production and maintain deployment history.

Phase 5 — Security and Reliability

- Use least-privilege IAM roles and separate deployment/runtime roles.
- Restrict security groups by tier; expose only ALB ports publicly.
- Encrypt EBS, RDS, logs, and supported storage.
- Enable backups, ALB/EC2 health checks, Auto Scaling, and controlled rollback.

Phase 6 — Observability and Release

- CloudWatch dashboards and alarms for ALB, EC2, ASG, RDS, CPU, memory, latency, and errors.
- Centralize application logs and define retention.
- Perform deployment smoke tests and rollback validation.
- Document operational runbook and release checklist.

EC2 Project Acceptance Criteria

- Application is reachable through the ALB using HTTPS.
- Frontend can call backend APIs and persist task data in RDS MySQL.
- EC2 instances are created through Terraform and can scale through the ASG.
- GitHub Actions validates every pull request and deploys approved changes from main.
- No long-lived AWS access keys are stored in GitHub.
- CloudWatch logs/metrics and alarms are available.
- A failed deployment can be rolled back using the documented release procedure.

3. Project 2 — 3-Tier Application on Amazon ECS

Objective: Deploy the same application as containers on Amazon ECS, preferably using ECS on Fargate to minimize host management. This project demonstrates container image lifecycle, ECS services/tasks, ALB integration, autoscaling, IAM task roles, and GitHub Actions CD.

Phase 1 — Containerization

- Create production-ready frontend and backend Dockerfiles.
- Use multi-stage builds where useful and run containers as non-root users.
- Add health checks and immutable version tags.
- Validate the stack locally with Docker Compose.

Phase 2 — AWS Foundation with Terraform

- Create VPC, multi-AZ subnets, security groups, ALB, ECS cluster, IAM roles, ECR repositories, and RDS MySQL.
- Keep ECS tasks and RDS in private subnets where architecture permits.
- Configure CloudWatch log groups and encrypted Terraform remote state.
- Use separate environment configuration for dev/stage/prod.

Phase 3 — ECS Service Deployment

- Create ECS task definitions for frontend and backend.
- Create ECS services behind the ALB with target-group health checks.
- Configure CPU/memory reservations, desired count, deployment settings, and autoscaling.
- Use Secrets Manager/Parameter Store references rather than embedding secrets in task definitions or Git.

Phase 4 — GitHub Actions CI/CD

- Pull request: unit tests, frontend build, backend checks, Terraform validation, Checkov, Trivy image scan, and code-quality scan.
- Merge to main: build immutable Docker images and push them to ECR.

- Update ECS task definitions and deploy the new image versions.
- Run post-deployment health checks; use GitHub Environment approval before production.

Phase 5 — Security and Reliability

- Use GitHub OIDC with scoped AWS IAM roles.
- Enable ECR image scanning and enforce vulnerability review gates.
- Use ECS task roles with only required AWS permissions.
- Use ALB health checks, ECS deployment circuit breaker/rollback, autoscaling, and RDS backups/encryption.

Phase 6 — Observability and Operations

- Send ECS application logs to CloudWatch Logs.
- Create dashboards and alarms for ECS service health, task count, CPU/memory, ALB latency/errors, and RDS.
- Track deployment status and application health.
- Document incident, rollback, scaling, and operational procedures.

ECS Project Acceptance Criteria

- Frontend and backend containers run successfully on ECS.
- ALB routes traffic to healthy ECS services.
- RDS MySQL is private and reachable only from the required application security group.
- Images are versioned and stored in ECR; production does not use mutable latest-only releases.
- GitHub Actions builds, scans, publishes, and deploys containers with controlled production approval.
- ECS services recover unhealthy tasks and support controlled rolling deployments.
- CloudWatch provides application and infrastructure visibility.

4. Standard GitHub Actions Pipeline

Stage	Process
1. Pull Request	Checkout → dependency install → unit tests → lint/build → Terraform fmt/validate → security scans.
2. Build	Build application artifacts/images; tag with commit SHA/version; do not rely on mutable latest tags.
3. Security	Trivy image/filesystem scan, Checkov IaC scan, code-quality scan, dependency checks; review high/critical findings.
4. Publish	Push approved container images to Amazon ECR for ECS; retain immutable tags.
5. Deploy Dev	Assume AWS role using GitHub OIDC → Terraform/app deployment → smoke tests.
6. Promote	Use GitHub Environment approval and protected branch/rules before production.
7. Production Deploy	Deploy using controlled rollout; monitor health checks and CloudWatch metrics.
8. Rollback	Re-deploy the last known-good immutable version/task definition or approved EC2 release.

5. Enterprise Repository Structure

app/

frontend/

backend/

infra/

terraform/

modules/

environments/dev/

environments/prod/

deploy/

ec2/

ecs/

.github/workflows/

ci.yml

deploy-ec2.yml

deploy-ecs.yml

tests/

docs/

6. Recommended AWS Controls

Area	Standard
Identity	IAM least privilege, separate runtime/deployment roles, GitHub OIDC, no long-lived CI credentials.
Networking	Multi-AZ VPC, private application/data tiers, restrictive security groups, controlled egress.
Secrets	AWS Secrets Manager or Parameter Store; never commit credentials or production secrets.
Encryption	TLS in transit; KMS-backed encryption for supported AWS storage/services.
CI/CD	Protected branches, PR checks, environment approvals, immutable release versions, rollback path.
Infrastructure	Terraform modules, remote encrypted state, plan/apply separation, code review for infrastructure changes.
Observability	CloudWatch logs, metrics, alarms, dashboards, defined retention and operational runbooks.
Cost	Right-size compute, use autoscaling, monitor RDS/ECS/EC2 utilization, and remove unused non-production resources.

7. 6- Hourly Execution Plan

Hour	Deliverable
Hr 1	Application foundation: React + FastAPI + MySQL, API contracts, health checks, local Docker Compose.
Hr 2	Terraform AWS foundation: VPC, subnets, security groups, ALB, RDS, IAM, remote state.
Hr 3	Project 1 EC2: launch template/ASG, application provisioning, CloudWatch, HTTPS, smoke tests.
Hr 4	Project 1 CI/CD: GitHub Actions, OIDC, security scans, deployment approvals, rollback.

Hr 5	Project 2 ECS: Docker hardening, ECR, ECS/Fargate, task definitions, services, ALB, autoscaling.
Hr 6	Project 2 CI/CD + hardening: image scanning, OIDC, deployment/rollback, monitoring, documentation, final demo.

8. Final Project Checklist

- Application runs locally and passes functional tests.
- Terraform provisions AWS infrastructure consistently.
- EC2 project demonstrates ALB + ASG + RDS + CloudWatch + GitHub Actions.
- ECS project demonstrates ECR + ECS/Fargate + ALB + RDS + CloudWatch + GitHub Actions.
- CI runs automatically on pull requests and protected branches.
- AWS access from GitHub uses OIDC and least-privilege IAM.
- Security and infrastructure scans are integrated into CI.
- Production secrets are managed outside Git.
- Monitoring, alarms, health checks, and rollback are tested.
- Architecture diagrams, README, runbooks, and deployment procedures are complete.